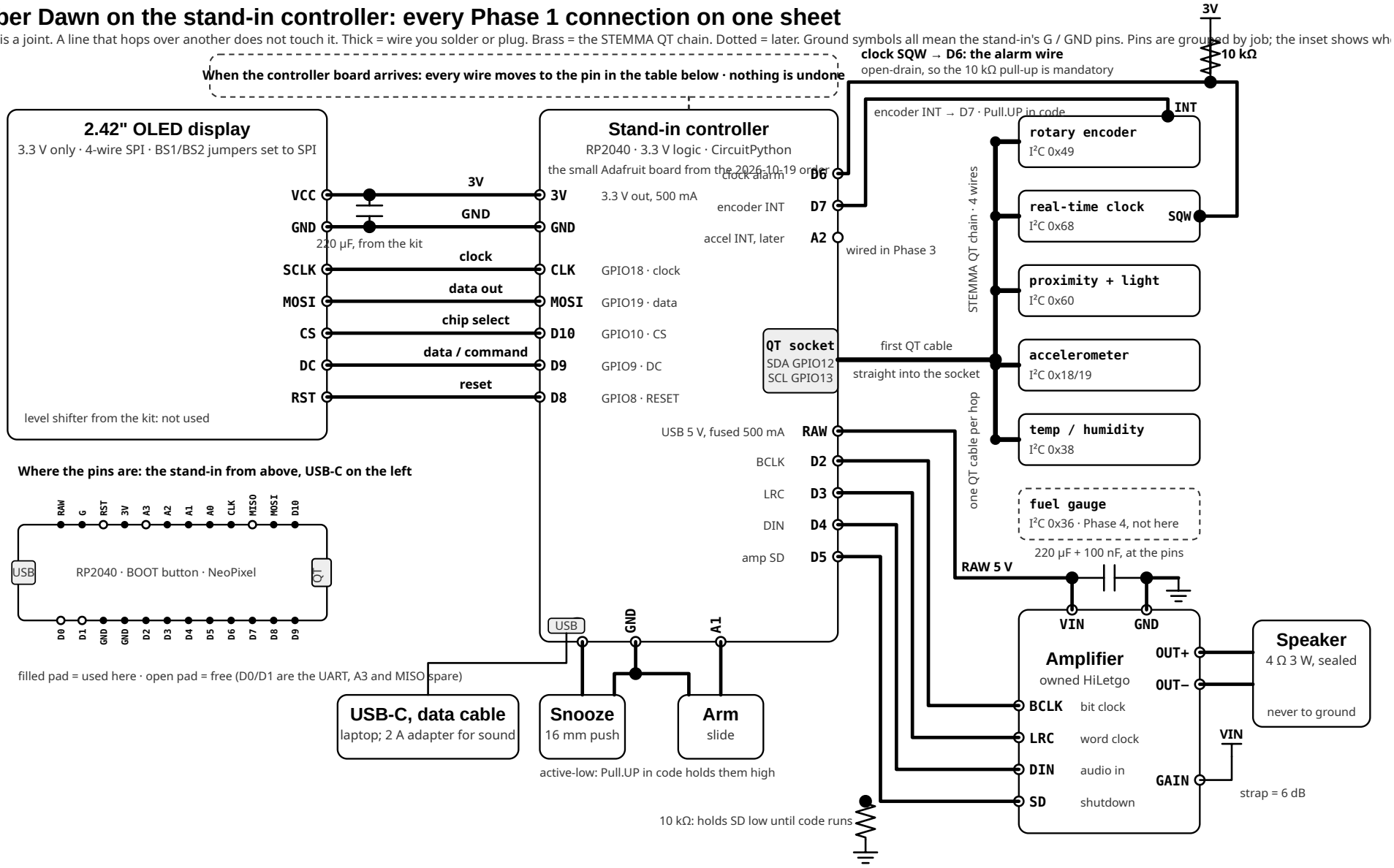
Cyber Dawn on the stand-in controller: every Phase 1 connection on one sheet

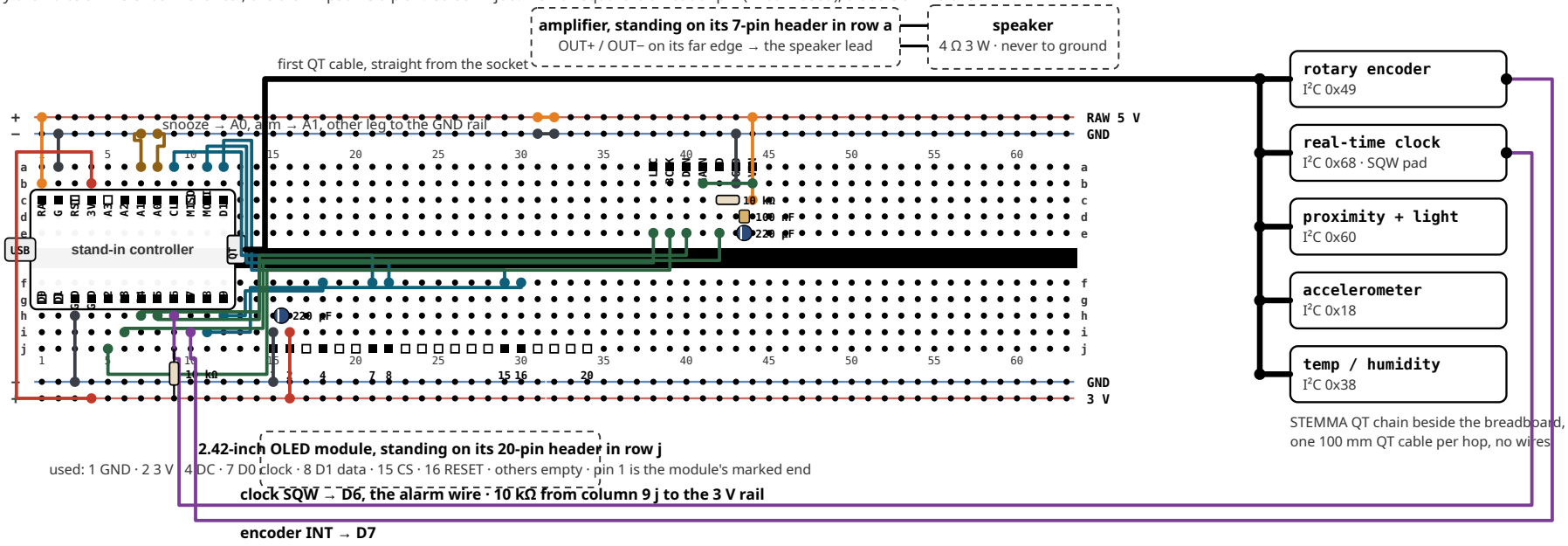
A dot is a joint. A line that hops over another does not touch it. Thick = wire you solder or plug. Brass = the STEMMA QT chain. Dotted = later. Ground symbols all mean the stand-in's G / GND pins. Pins are grouped by job; the inset shows wh



Pin facts from the board's Adafruit guide and its CircuitPython pins.c, read 2026-10-31. Not built. The pin audit and the line-clearance check in gen/wiring_standin.py pass.

The same wiring on an 830-point breadboard: where every pin and every wire end goes

Columns are numbered as on the board. The stand-in straddles the trench, columns 1-12: top-edge pins in row c, bottom-edge pins in row g.
Only the holes a wire ends in are real; the drawn path is a plan. Colour = job. A small square is a header pin (filled = used); a dot is a wire end.



RAW 5 V (amp only) · 3 V · GND · display SPI · amp I²S + strap · alarm + encoder INT · switch leads

Rails: top pair = RAW 5 V and GND, bottom pair = 3 V and GND. The 3 V rail is fed by the one wire that goes around the USB end of the stand-in.
The two top rails are bridged across the board's middle split. The OLED's 220 µF sits at its pins 1-2; the amp's 220 µF + 100 nF across GND-VIN.

CYBER DAWN · PHASE 1 ON THE STAND-IN CONTROLLER

The bench clock on the board you already have

The whole Phase 1 wiring moved onto the small Adafruit RP2040 board from the 2026-10-19 order, so the alarm clock can be built and run now, before the controller board arrives. One sheet, the pin map with the pin each wire moves to later, the same wiring laid out hole by hole on an 830-point breadboard (added the evening of 2026-10-31), and the few things the stand-in cannot do.
Written 2026-10-31. Companion to `docs/DAY0_TEST.md` and `pages/cyber-dawn-wiring.html`.

What this sheet is

The controller board is still in transit, and the stand-in controller, the small Adafruit RP2040 board that came free with the 2026-10-19 order, has every pin the Phase 1 bench needs: a STEMMA QT socket for the chain, a hardware SPI for the OLED display, three free GPIOs in a row for the amplifier's I²S, and spare inputs for the clock alarm, the encoder, the snooze button and the arm switch. So the whole bench clock can be built and run now, in CircuitPython, and when the controller board lands each wire moves to the pin in the table's last column. Nothing gets desoldered: the OLED header and its capacitor, the two resistors, the switch leads and the speaker lead all stay as they are.

This sheet extends the Day 0 plan: Day 0's chain, OLED and amp steps used exactly these pins, and this page adds the amp shutdown, the clock alarm wire, the encoder interrupt and the two switches so that the alarm itself can be tested.

THREE RULES, UNCHANGED

Every logic line is 3.3 V. The display, the clock and every sensor run from the stand-in's 3V pin or its QT socket. RAW is 5 V and goes to the amplifier VIN and nowhere else.

One power source: the USB-C cable. There is no battery and no charger board on this sheet. Use a data cable (a charge-only cable gives no CIRCUITPY drive).

The amplifier's outputs are a bridge. OUT+ and OUT− are never ground, never a scope clip, never brass.

The pin map

JOB	STAND-IN PIN	GPIO	CIRCUITPYTHON	CONTROLLER BOARD, LATER	NOTE
I²C SDA, the STEMMA QT chain	QT socket	12	board.SDA (board.STEMMA_I2C())	GPIO8	No wire: the first QT cable plugs straight into the socket.
I²C SCL	QT socket	13	board.SCL	GPIO9	
3.3 V to the display and the chain	3V			3V3	The socket feeds the chain; one wire feeds the display. 500 mA budget.

Ground	G, GND, GND			GND	Three ground pins; any one, all are the same copper.
Display clock	CLK	18	board.CLK (also board.SCK)	GPIO12	The hardwar SPI0 clock.
Display data	MOSI	19	board.MOSI	GPIO11	
Display chip select	D10	10	board.D10	GPIO10	
Display data / command	D9	9	board.D9	GPIO6	
Display reset	D8	8	board.D8	GPIO7	
Amp bit clock	D2	2	board.D2	GPIO14	I²S on this chip is a PIO program word select must be the GPIC right after bit clock, so D2 and D3.
Amp word clock	D3	3	board.D3	GPIO15	
Amp data	D4	4	board.D4	GPIO16	
Amp shutdown	D5	5	board.D5	GPIO17	10 kΩ to ground at the amp, as on the controlle board.
Amp 5 V	RAW			5V	USB 5 V after the board's 500 mA fuse. Bench volume only; see the limits.

Clock alarm (SQW)	D6	6	board.D6	GPIO4	10 kΩ to 3V: the output is open-drain.
Encoder INT	D7	7	board.D7	GPIO1	Pull.UP i code.
Snooze button	A0	26	board.A0	GPIO18	Other leg to GND; Pull.UP i code.
Arm switch	A1	27	board.A1	GPIO21	Other leg to GND; Pull.UP i code.
Accelerometer INT, Phase 3	A2	28	board.A2	GPIO5	Not wired yet.
Free	D0, D1, A3, MISO				D0/D1 are the UART; leave them for a serial probe.

Declared in code, this is the top of `code.py`:

```
import board, busio, digitalio
from digitalio import DigitalInOut, Direction, Pull

i2c = board.STEMMA_I2C()                    # the chain, through the socket

spi = busio.SPI(board.CLK, board.MOSI)      # the display, 4-wire SPI
OLED_CS, OLED_DC, OLED_RST = board.D10, board.D9, board.D8

I2S_BCLK, I2S_LRC, I2S_DIN = board.D2, board.D3, board.D4
amp_sd = DigitalInOut(board.D5); amp_sd.direction = Direction.OUTPUT; amp_sd.value = False

def input_pullup(p):
    d = DigitalInOut(p); d.direction = Direction.INPUT; d.pull = Pull.UP; return d
clock_alarm = input_pullup(board.D6)        # low when Alarm 1 fires (the 10 kΩ does the real work)
encoder_int = input_pullup(board.D7)        # low on any turn or press
snooze      = input_pullup(board.A0)        # low while pressed
armed       = input_pullup(board.A1)        # low = armed, if the switch is wired that way round
```

1 · The chain: one cable into the socket

The QT-to-header cable stays in the bag. The first 100 mm QT cable plugs into the stand-in's socket (SDA GPIO12, SCL GPIO13) and the five boards chain from there in any order; the sheet keeps the Day 0 order so the scan list matches:

rotary encoder 0x49, real-time clock 0x68, proximity and light sensor 0x60, accelerometer 0x18, temperature and humidity sensor 0x38. The fuel gauge is a Phase 4 part and is not on the bench; five boards plus the display is also the comfortable limit of the stand-in's 500 mA regulator.

2 · The display: the Day 0 wiring, kept

VCC to 3V, GND to GND, then CLK, MOSI, D10 for chip select, D9 for data/command, D8 for reset. The kit's 220 µF sits across VCC and GND at the module and the level shifter stays in the bag. CLK and MOSI are the hardware SPI0 pins, so `busio.SPI(board.CLK, board.MOSI)` is the fast path. Photograph the BS1/BS2 jumpers before soldering the header, as the Day 0 plan says.

3 · Audio: four wires, RAW, and a fuse to respect

BCLK on D2, LRC on D3, DIN on D4, SD on D5 with 10 kΩ to ground at the amp; GAIN strapped to VIN for the 6 dB ceiling; VIN from RAW; 220 µF and 100 nF across VIN and GND at the amp's pins. The RP2040 makes I^2S with a PIO program and CircuitPython's `audiobusio.I2SOut` requires the word-select pin to be the GPIO right after the bit-clock pin, which is why D2 and D3 are fixed and D4 is the data.

The fuse. RAW is USB 5 V after the stand-in's 500 mA polyfuse and its protection diode. The amplifier at 6 dB into 4 Ω peaks near 0.9 A, which is why the controller board's design uses a 2 A adapter. On the stand-in keep the volume at bench level (the Day 0 tone at 6000 of 32767 is right) and treat a sudden silence after a loud passage as the fuse resetting, not a fault. There is a jumper on the back of the board that bypasses the fuse for 2 A; leave it alone, the full-volume alarm test belongs on the controller board.

4 · The alarm wire, the encoder, and the two switches

Clock SQW to D6 with 10 kΩ to 3V. The clock pulls the line low when Alarm 1 matches. The output is open-drain, so without the resistor the line floats and the alarm never fires: that resistor is the whole test. On the stand-in any GPIO can wake CircuitPython from light or deep sleep with `alarm.pin.PinAlarm(board.D6, value=False)`, so the deep-sleep wake test can be run here too.

Encoder INT to D7, pulled up in code; it pulses low on any turn or press so the loop can skip polling. **Snooze on A0, arm on A1**, each other leg to GND, both pulled up in code. A0 to A3 are the ADC-capable pins, but as plain digital inputs they are no different from the D pins, and using them keeps D0/D1 free for a serial probe.

What the stand-in cannot do

- [ ] Run the ESP-IDF firmware in `firmware/`: it is an ESP32-S3 program. The stand-in runs CircuitPython, so what gets proven here is the hardware and the alarm logic, not the firmware.
- [ ] Full alarm volume: the 500 mA fuse on RAW, above.
- [ ] Wi-Fi: no radio. Time is set from the console and kept by the clock's coin cell.
- [ ] Feed a sixth QT board or anything else from 3V: the regulator is rated 500 mA peak and the display, the chain and the chip already use a share of it.

Before power, with the meter

- [ ] 3V to GND and RAW to GND: no steady beep. A chirp that stops is a capacitor.
- [ ] OUT+ and OUT− to GND and to each other: silent, or a few ohms through the speaker only.
- [ ] Clock SQW to 3V: about 10 kΩ. Open means the pull-up is missing and the alarm will never fire.
- [ ] Amp SD to GND: about 10 kΩ. Near 1 MΩ or open means the pull-down is missing.
- [ ] Each display signal pin to its stand-in pin: beep. Adjacent display pins to each other: silent.

The same wiring on a breadboard

What. One of the owned 830-point breadboards carries the whole test. The stand-in's two pin rows are 0.6 inch apart, so it straddles the trench: top-edge pins (RAW, G, RST, 3V, A3 to A0, CLK, MISO, MOSI, D10) in row `c`, bottom-edge pins (D0, D1, GND, GND, D2 to D9) in row `g`, columns 1 to 12, USB-C hanging off the left end. Holes `a` - `b` above it and `h` - `j` below it stay free for wires. The OLED module's **20-pin** header goes into row `j`, columns 15 to 34, the module standing up off the bottom edge; the amp's 7-pin header into row `a`, columns 38 to 44, standing up off the top edge. The five QT boards never touch the breadboard: the first QT cable leaves the stand-in's socket and the chain lies beside the board.

Rails. Top pair: RAW 5 V (red, fed from column 1) and GND (blue, from column 2). Bottom pair: GND (blue, from the bottom-edge GND in column 3) and 3 V (red). The 3 V rail is the one awkward wire: 3V is a top-edge pin, so its jumper leaves column 4 and goes around the USB end of the stand-in to the bottom red rail. The owned boards ship with the rails split at the middle; the two top rails are used on both sides of the split, so bridge each with a short jumper at columns 31-32. The bottom rails are only used left of the split.

Order of work, so the meter checks below stay meaningful: (1) stand-in in, nothing else, power it, see CIRCUITPY; (2) the three rail feeds and the two split bridges, meter 3 V and 5 V against GND; (3) the chain by QT cable, scan; (4) OLED header, its 220 µF, two power wires, five signal wires; (5) amp header, its 10 kΩ, two capacitors, GAIN strap, two power wires, four signal wires; (6) the SQW wire and its 10 kΩ, the INT wire; (7) the two switches on flying leads. Column numbers in the drawing are the ones printed on the board.

TWO FACTS THAT DECIDE THIS DRAWING

The OLED header is 20 pins, not 7. Adafruit's guide for the 1.5"/2.4" 128×64 module family numbers them 1 to 20 from the left: 1 ground, 2 3 V in, 4 DC, 7 Data0 (the SPI clock), 8 Data1 (the SPI data), 15 CS, 16 RESET; 5-6 and 9-14 are the parallel-bus pins, never connected in SPI; 20 is the frame ground. The module may ship in 8-bit mode; the solder-jumper photo decides whether it is already SPI. If it is not, the Day 0 plan says stop, and the display waits for the controller board.

The stand-in's fuse. RAW is USB 5 V after a 500 mA fuse. The amp alone can pull 0.9 A peaks at 6 dB into 4 Ω, so keep the test tone quiet and expect the fuse, not a wiring fault, if sound cuts out after a loud passage.

Built by `docs/pages/gen/wiring_standin.py`. Pin facts from the board's Adafruit guide and its CircuitPython board definition, read 2026-10-31; not verified against hardware. Nothing on this page has been built.